BACK MATTER

Afterword

The book has made its case. What's left is to say plainly what it doesn't cover, and what would have to be true for the case to be wrong.

By now you've either tried writing a vision statement for a feature your team was about to hand an AI assistant, or you've filed this book's exercises under someone else's project. Either way, the argument is finished. Eleven chapters built one example, a clinic group's booking feature, and stress-tested the method against a second, a bank's 1980s COBOL ledger, to show what a specification is for and what happens to a system that never had one written down. What's left in these last few pages is smaller: what the book actually claimed, what it left out on purpose, and the honest possibility that the whole thing rests on an assumption that doesn't hold.

What the argument comes down to

Set the examples aside and the claim underneath them is short. Deciding what the code should do has always cost more than typing it did, and AI made that visible only by removing the part that used to disguise it: writing the code doubled as thinking the problem through, for as long as a person had to write it by hand.

A specification, written down and kept current, holds that decision longer than the codebase built from it, because the technology underneath changes before the business logic does. COBOL gave way to Java in the bank's ledger. The rule that a ledger mismatch holds the account for manual review did not. Structure is what lets AI implement a decision instead of inventing one: vision, requirements, use cases, a domain model, architecture, skills. Structure is also what lets a human check the result against something more specific than a feeling.

This argument moves judgment in engineering away from syntax, which nobody has to type by hand anymore, and back toward the question that was always hard: what should this system do, and for whom.

Where this book stops short

A methodology book has to stop somewhere. This one stopped short of several subjects that deserve a book of their own rather than a section tacked onto this one.

Ethics and accountability: this book has assumed a specification with one accountable owner who stays responsible for it, and an implementation reviewed by a human before it ships. That's a workable model at the scale of one clinic group's booking feature. It says nothing about what changes once an organization is implementing hundreds of specifications a week and no single person has read all of them. This book doesn't answer that question.

Security and trust: generated code needs the same scrutiny as any other code, and in places more. An AI assistant filling in a validation rule it inferred, rather than one the domain model specified, is a plausible way for a vulnerability to enter a system that looks entirely conventional on review. This book mentions that risk without building the review discipline it requires.

Organizational change: adopting this discipline changes who signs off on a specification, what a standup reports on, and how a team that has always been measured on shipped code reacts to being asked to spend a week on a domain model instead. Those are people problems more than technical ones, and this book, being a technical methodology, was never equipped to solve them.

Tooling: git, diff, and code review all grew up around code. Nothing described here has an equivalent that's mature and widely adopted. A skill from Chapter 7 ends up as a document in a repository, the closest available container rather than the right one.

Regulatory compliance: the bank's ledger, this book's own running example, sits inside documentation and audit-trail requirements that a regulator enforces directly. Whether a specification recovered the way Chapter 9 describes would satisfy a specific regulator is a question for someone who knows that regulator.

What if the premise is wrong

Everything above assumes a bet pays off: that a specification is cheaper to keep current than code is, and that AI can be trusted to implement from one reliably enough that the review this book describes stays lighter than a full rewrite. That bet could be wrong.

Specifications could drift the same way code comments do, accurate on the day they're written and stale within two sprints, because nothing enforces keeping them current the way a compiler enforces syntax. A use case that says a patient books one slot with one eligible doctor is only as good as the discipline that updates it the day that rule changes, and nothing in this book manufactures that discipline out of nothing.

An organization could adopt every layer this book describes, vision through skills, and still end up maintaining two things instead of one: the specification, and the code that drifted away from it. That would be a worse position than maintaining code alone. This book cannot rule out that failure. It shows up exactly where the discipline is weakest, usually in the specification nobody has looked at since the meeting where it was written.

Chapter 8's advice on testing and Chapter 9's advice on legacy recovery both apply here just as much as they applied there: start on one feature, keep the specification and the code in view together, and treat a specification you've stopped trusting as a signal to repair the discipline. A pilot that fails this way is a finding, and reporting it honestly to the rest of the organization is part of the discipline working as intended.

Why this, why now, and where that leaves you

None of the ideas in this book are new. Engineers have written some version of a specification for decades, and the Foreword traces that lineage. What changed is the arithmetic. Hours not spent specifying the work show up later, as review time and rework, on code nobody can be sure was built for the right business.

Whether that arithmetic holds up in your organization depends on how often your requirements actually change, how much of your domain currently lives only in someone's head, and whether your team keeps a specification current once the novelty of a new practice wears off, the way it does for any practice.

What this book can offer is a specific, checkable test. The Schedule Appointment use case in Chapter 3, the domain model in Chapter 4, and the Spring Boot skill in Chapter 7 are things you can write for your own feature this week. If the specification you produce lets an AI assistant implement that feature correctly on something close to the first pass, the argument in this book has done its job for you. If it doesn't, you'll know quickly, and you'll know exactly which layer to blame.

That's the test this book leaves you, and it's a more useful measure than whether the argument was persuasive.

Afterword ends here.